

Một số định nghĩa:

- ☐ **Database:** Một cơ sở dữ liệu là một tập hợp các bảng dữ liệu, với dữ liệu có liên quan.
- ☐ **Table - Bảng dữ liệu:** Một bảng là một ma trận dữ liệu. Một bảng trong một cơ sở dữ liệu trông giống như một bảng tính đơn giản.
- ☐ **Cột:** Một cột chứa cùng một kiểu dữ liệu, ví dụ như tên khách hàng.
- ☐ **Hàng:** Một hàng (row, entry, record) là một nhóm dữ liệu có liên quan.
- ☐ **Redundancy:** (có thể hiểu là dữ liệu dự phòng) Dữ liệu được lưu giữ hai lần, để làm cho hệ thống nhanh hơn.
- ☐ **Primary Key:** Một Primary Key (Khóa chính) là duy nhất. Một giá trị key không thể xuất hiện hai lần trong một bảng. Với một key, bạn có thể tìm thấy phần lớn trên một hàng.
- ☐ **Foreign Key:** Bạn tưởng tượng về Foreign Key như là cái ghim liên kết giữa hai bảng.
- ☐ **Compound Key:** Một Compound Key (hay composite key) là một key mà gồm nhiều cột, bởi vì một cột là không duy nhất.
- ☐ **Index:** Một chỉ mục trong một cơ sở dữ liệu tương tự như chỉ mục trong một cuốn sách.
- ☐ **Referential Integrity:** Đảm bảo rằng một giá trị Foreign Key luôn luôn trỏ tới một hàng đang tồn tại.

Các kiểu dữ liệu

MySQL sử dụng nhiều kiểu dữ liệu, được chia thành 3 loại: kiểu số, kiểu date/time, và kiểu xâu ký tự (string).

1.1 Kiểu dữ liệu số trong MySQL

MySQL sử dụng tất cả các kiểu dữ liệu số theo chuẩn ANSI SQL, vì thế nếu bạn đã làm quen với một hệ thống cơ sở dữ liệu khác, thì bạn sẽ thấy những định nghĩa này là khá thân thuộc với bạn khi học về MySQL. Dưới đây liệt kê các kiểu dữ liệu số phổ biến và miêu tả của chúng:

- ☐ **INT** - Một số nguyên với kích cỡ thông thường, có thể là signed hoặc unsigned. Nếu có dấu, thì dãy giá trị có thể là từ -2147483648 tới 2147483647, nếu không dấu thì dãy giá trị là từ 0 tới 4294967295. Bạn có thể xác định một độ rộng lên tới 11 chữ số.
- ☐ **TINYINT** - Một số nguyên với kích cỡ rất nhỏ, có thể là signed hoặc unsigned. Nếu có dấu, thì dãy giá trị có thể là từ -128 tới 127, nếu không dấu thì dãy giá trị là từ 0 tới 255. Bạn có thể xác định một độ rộng lên tới 4 chữ số.

- **SMALLINT** - Một số nguyên với kích cỡ nhỏ, có thể là signed hoặc unsigned. Nếu có dấu, thì dãy giá trị có thể là từ -32768 tới 32767, nếu không dấu thì dãy giá trị là từ 0 tới 65535. Bạn có thể xác định một độ rộng lên tới 5 chữ số.
- **MEDIUMINT** - Một số nguyên với kích cỡ trung bình, có thể là signed hoặc unsigned. Nếu có dấu, thì dãy giá trị có thể là từ -8388608 tới 8388607, nếu không dấu thì dãy giá trị là từ 0 tới 16777215. Bạn có thể xác định một độ rộng lên tới 9 chữ số.

- ❑ **BIGINT** - Một số nguyên với kích cỡ lớn, có thể là signed hoặc unsigned. Nếu có dấu, thì dãy giá trị có thể là từ -9223372036854775808 tới 9223372036854775807, nếu không dấu thì dãy giá trị là từ 0 tới 18446744073709551615. Bạn có thể xác định một độ rộng lên tới 20 chữ số.
- ❑ **FLOAT(M,D)** - Một số thực dấu chấm động không dấu. Bạn có thể định nghĩa độ dài hiển thị (M) và số vị trí sau dấu phẩy (D). Điều này là không bắt buộc và sẽ có mặc định là 10,2: với 2 là số vị trí sau dấu phẩy và 10 là số chữ số (bao gồm các phần thập phân). Phần thập phân có thể lên tới 24 vị trí sau dấu phẩy đối với một số FLOAT.
- ❑ **DOUBLE(M,D)** - Một số thực dấu chấm động không dấu. Bạn có thể định nghĩa độ dài hiển thị (M) và số vị trí sau dấu phẩy (D). Điều này là không bắt buộc và sẽ có mặc định là 16,4: với 4 là số vị trí sau dấu phẩy và 16 là số chữ số (bao gồm các phần thập phân). Phần thập phân có thể lên tới 53 vị trí sau dấu phẩy đối với một số DOUBLE. REAL là đồng nghĩa với DOUBLE.
- ❑ **DECIMAL(M,D)** - Một kiểu khác của dấu chấm động không dấu. Mỗi chữ số thập phân chiếm 1 byte. Việc định nghĩa độ dài hiển thị (M) và số vị trí sau dấu phẩy (D) là bắt buộc. NUMERIC là một từ đồng nghĩa cho DECIMAL.

1.2 Kiểu dữ liệu Date và Time trong MySQL

Kiểu dữ liệu Date và Time được phân loại thành:

- ❑ **DATE** - Một date trong định dạng YYYY-MM-DD, giữa 1000-01-01 và 9999-12-31. Ví dụ, ngày 25 tháng 12 năm 2015 sẽ được lưu ở dạng 2015-12-25.
- ❑ **DATETIME** - Một tổ hợp Date và Time trong định dạng YYYY-MM-DD HH:MM:SS, giữa 1000-01-01 00:00:00 và 9999-12-31 23:59:59. Ví dụ, 3:30 chiều ngày 25 tháng 12, năm 2015 sẽ được lưu ở dạng 2015-12-25 15:30:00.
- ❑ **TIMESTAMP** - Một Timestamp từ giữa nửa đêm ngày 1/1/1970 và 2037. Trông khá giống với định dạng DATETIME trước, khác biệt ở chỗ không có dấu gạch nối giữa các số. Ví dụ, 3:30 chiều ngày 25 tháng 12, năm 2015 sẽ được lưu dưới dạng 20151225153000 (YYYYMMDDHHMMSS).
- ❑ **TIME** - Lưu time trong định dạng HH:MM:SS.
- ❑ **YEAR(M)** - Lưu 1 năm trong định dạng 2 chữ số hoặc 4 chữ số. Nếu độ dài được xác định là 2 (ví dụ: YEAR(2)), YEAR có thể từ 1970 tới 2069 (70 tới 69). Nếu độ dài được xác định là 4, YEAR có thể từ 1901 tới 2155. Độ dài mặc định là 4.

1.3 Các thao tác cơ bản với cơ sở dữ liệu

1. Tạo database

Cú pháp:

```
CREATE DATABASE Ten_co_so_du_lieu;
```

Ví dụ:

```
CREATE DATABASE sinhvien;
```

2. Xóa database

Cú pháp:

```
DROP DATABASE ten_co_so_du_lieu;
```

Ví dụ:

```
DROP DATABASE sinhvien;
```

3. Tạo bảng

Cú pháp

```
CREATE TABLE ten_bang (ten_cot kieu_du_lieu);
```

Ví dụ

Tạo một bảng có tên là `sinhvienk61` với các trường `mssv`, `ho`, `ten`, `tuoi`, `diemthi` trong cơ sở dữ liệu `sinhvien`:

```
CREATE TABLE sinhvienk61
(
    mssv INT NOT NULL
    AUTO_INCREMENT, ho VARCHAR(255)
    NOT NULL, ten VARCHAR(255) NOT
    NULL,
    tuoi INT NOT NULL,
    diemthi FLOAT(4,2) NOT NULL,
    PRIMARY KEY (mssv)
);
```

Trong đó:

- ☐ Thuộc tính **NOT NULL** của trường đang được sử dụng bởi vì chúng ta không muốn trường này là NULL. Vì thế, nếu người dùng cố gắng tạo một bản ghi có giá trị NULL, thì MySQL sẽ báo lỗi.
- ☐ Thuộc tính **AUTO_INCREMENT** nói cho MySQL tự động tăng khóa chính và thêm giá trị có sẵn tiếp theo tới trường `id`.
- ☐ Từ khóa **PRIMARY KEY** được sử dụng để định nghĩa một cột là PRIMARY KEY (khóa chính). Bạn có thể sử dụng nhiều cột phân biệt nhau bởi dấu phẩy để định nghĩa một PRIMARY KEY.
- ☐ Nếu bạn có nhiều cơ sở dữ liệu, thì để tạo bảng `sinhvienk60` có trong cơ sở dữ liệu `sinhvien` thì trước hết bạn phải chọn cơ sở dữ liệu đó với lệnh `USE`.
- ☐ Các kiểu dữ liệu (xem phần sau)

```
SHOW COLUMNS FROM sinhvienk61;
```

4. Thay đổi bảng

Lệnh **ALTER** trong MySQL là thực sự hữu ích khi bạn muốn thay tên cho một bảng, cho bất kỳ trường nào hoặc nếu bạn muốn thêm hoặc xóa một cột đang tồn tại trong một bảng.

Ví dụ

```
//Lua chon co so du lieu
USE sinhvien;
//Tao bang hocphik61

CREATE TABLE hocphik61
(
    ten VARCHAR(40),
    hocphi INT
);
```

//Hiển thị tất cả các cột trong bảng hocphik61

SHOW COLUMNS FROM hocphik61;

//Kết quả là:

Field	Type	Null	Key	Default	Extra
ten	varchar(40)	YES		NULL	
hocphi	int(11)	YES		NULL	

Giả sử bạn muốn xóa một cột đang tồn tại *ten* từ bảng MySQL trên, thì bạn sử dụng mệnh đề **DROP** cùng với lệnh **ALTER** như sau:

Xóa một cột đang tồn tại:

ALTER TABLE hocphik61 DROP ten;

Thêm một cột vào bảng:

ALTER TABLE hocphik61 ADD ten VARCHAR(40);

Sau khi thông báo lệnh này, bảng hocphik61 sẽ chứa cùng hai cột như nó đã chứa khi bạn lần đầu tạo bảng đó, nhưng sẽ không có cùng cấu trúc. Đó là bởi vì các cột mới được thêm vào phần cuối bảng theo mặc định. Vì thế, ngay cả khi lúc ban đầu cột **ten** là cột đầu tiên trong bảng hocphik61, thì bây giờ nó là cột cuối cùng.

SHOW COLUMNS FROM hocphik61;

Field	Type	Null	Key	Default	Extra
hocphi	int(11)	YES		NULL	
ten	varchar(40)	YES		NULL	

Để chỉ rằng bạn muốn một cột tại một vị trí cụ thể bên trong một bảng, hoặc sử dụng **FIRST** để làm nó trở thành cột đầu tiên hoặc **AFTER ten_cot** để chỉ rằng cột mới nên được đặt sau cột *ten_cot*. Bạn thử các lệnh **ALTER TABLE** sau, sử dụng **SHOW COLUMNS** sau mỗi lệnh để xem hiệu quả của mỗi lệnh đó:

ALTER TABLE hocphik61 DROP ten;

ALTER TABLE hocphik61 ADD ten VARCHAR(40) FIRST; //thêm cột **ten** thành cột đầu tiên

ALTER TABLE hocphik61 DROP ten;

ALTER TABLE hocphik61 ADD ten VARCHAR(40) AFTER hocphi; //thêm cột **ten** sau cột **hocphi**

FIRST và AFTER specifier chỉ làm việc với mệnh đề ADD. Điều này nghĩa là nếu bạn muốn tái định vị một cột đang tồn tại bên trong một bảng, đầu tiên bạn phải DROP nó và sau đó ADD nó tại vị trí mới.

Để thay đổi một định nghĩa cột, sử dụng mệnh đề **MODIFY** hoặc **CHANGE** cùng với lệnh

ALTER. Ví dụ, để thay đổi cột *ten* từ **VARCHAR(40)** thành **VARCHAR(20)**, bạn sử dụng:

```
ALTER TABLE hocphik61 MODIFY ten VARCHAR(20);
```

Với CHANGE, cú pháp có hơi chút khác biệt. Sau từ khóa CHANGE, bạn xác định cột bạn muốn thay đổi, sau đó xác định định nghĩa mới, bao gồm tên mới của nó.

```
ALTER TABLE hocphik61 CHANGE ten hoten VARCHAR(60);
```

Nếu bây giờ bạn sử dụng CHANGE để chuyển đổi hoten từ VARCHAR(60) về VARCHAR(40) mà không thay đổi tên cột, lệnh sẽ là:

```
ALTER TABLE hocphik61 CHANGE hoten hoten VARCHAR(40);
```

Tác động của ALTER TABLE trên các giá trị NULL và DEFAULT

Khi bạn MODIFY hoặc CHANGE một cột, bạn cũng có thể xác định có hay không cột đó có thể chứa các giá trị NULL và giá trị DEFAULT của nó là gì. Thực tế, nếu bạn không làm điều này, MySQL sẽ tự động gán các giá trị cho các thuộc tính này.

Trong ví dụ sau, cột NOT NULL sẽ có giá trị mặc định là 4000000.

```
ALTER TABLE hocphik61  
    MODIFY hocphi BIGINT NOT NULL DEFAULT 4000000;
```

Nếu bạn không sử dụng lệnh trên, thì MySQL sẽ điền các giá trị NULL vào tất cả các cột.

Thay đổi giá trị DEFAULT của một cột trong MySQL

Bạn có thể thay đổi một giá trị mặc định cho bất kỳ cột nào bởi sử dụng lệnh ALTER. Bạn thử ví dụ sau:

```
ALTER TABLE hocphik61 ALTER hocphi SET DEFAULT 3000000;
```

```
//hiển thị các cột  
SHOW COLUMNS FROM hocphik61;
```

```
//kết quả là:
```

Field	Type	Null	Key	Default	Extra
ten	varchar(40)	YES		NULL	
hocphi	int(11)	YES		3000000	

Bạn có thể xóa ràng buộc DEFAULT từ bất kỳ cột nào bởi sử dụng mệnh đề DROP cùng với lệnh ALTER.

```
ALTER TABLE hocphik61 ALTER hocphi DROP DEFAULT;
```



```
//hien thi cac cot  
SHOW COLUMNS FROM hocphik61;
```

//ket qua la:

Field	Type	Null	Key	Default	Extra
ten	varchar(40)	YES		NULL	
hocphi	int(11)	YES		NULL	

Thay tên cho bảng trong MySQL

Để thay tên cho một bảng, sử dụng tùy chọn **RENAME** của lệnh ALTER TABLE. Bạn xem ví dụ sau để đổi tên hocphik61 thành hocphik62.

ALTER TABLE hocphik61 RENAME TO hocphik62;

5. Xóa bảng

Cú pháp

DROP TABLE ten_bang;

Ví dụ

DROP TABLE sinhvienk61;

6. Chèn dữ liệu vào một bảng

Cú pháp

INSERT INTO ten_bang (truong1, truong2,...truongN)
VALUES
(giatri1, giatri2,...giatriN);

Ví dụ

INSERT INTO sinhvienk61 (ho, ten, diemthi)
VALUES ("Dinh Van", "Cao", 8);

INSERT INTO sinhvienk61 (ho, ten, diemthi)
VALUES ("Nguyen Van", "Thanh", 9);

INSERT INTO sinhvienk61 (ho, ten, diemthi)
VALUES ("Nguyen Hoang", "Manh", 7.5);

INSERT INTO sinhvienk61 (ho, ten, diemthi)
VALUES ("Tran Van", "Nam", 10);

7. Xem dữ liệu trong một bảng

Cú pháp

SELECT

```
[FROM table_references
[PARTITION partition_list]
[WHERE where_condition]
[GROUP BY {col_name | expr | position}
[ASC | DESC], ... [WITH ROLLUP]]
[HAVING where_condition]
[ORDER BY {col_name | expr | position}
[ASC | DESC], ...]
[LIMIT {[offset,] row_count | row_count OFFSET offset}]
[PROCEDURE procedure_name(argument_list)] [INTO
OUTFILE 'file_name'
[CHARACTER SET charset_name]
export_options
| INTO DUMPFILE 'file_name'
| INTO var_name [, var_name]]
[FOR UPDATE | LOCK IN SHARE MODE]]
```

Ví dụ

Select * from sinhvienK61;

8. Cập nhật dữ liệu trong bảng

Dữ liệu đang tồn tại trong một bảng MySQL có thể cần được sửa đổi. Bạn có thể thực hiện điều này bởi sử dụng lệnh **UPDATE** trong SQL. Lệnh này sẽ sửa đổi bất kỳ giá trị trường nào trong bất cứ bảng MySQL nào.

Cú pháp

Dưới đây là cú pháp SQL chung của lệnh UPDATE để sửa đổi dữ liệu trong bảng MySQL:

```
UPDATE ten_bang SET truong1=giaTri_moi_1, truong2=giaTri_moi_2
[MệnhĐề WHERE];
```

- Bạn có thể cập nhật một hoặc nhiều trường.
 - Bạn có thể xác định bất kỳ điều kiện nào bởi sử dụng mệnh đề WHERE.
 - Bạn có thể cập nhật các giá trị trong một bảng đơn tại một thời điểm.
- Mệnh đề WHERE là hữu ích khi bạn muốn cập nhật các hàng đã chọn trong một bảng.

Cập nhật dữ liệu trong MySQL

Bạn sử dụng lệnh UPDATE với mệnh đề WHERE trong SQL để cập nhật dữ liệu đã chọn vào trong bảng MySQL.

Ví dụ

Ví dụ sau sẽ cập nhật bản ghi có **mssv** là 3 trong trường **ten** trong bảng **sinhvienk60**:

```
UPDATE sinhvienk61 SET ten="Huong" WHERE mssv=3;
```

9. Xóa dữ liệu trong bảng

Nếu bạn muốn xóa một bản ghi từ bất cứ bảng MySQL nào, thì bạn có thể sử dụng lệnh **DELETE FROM** trong SQL.

Cú pháp

Cú pháp SQL chung của lệnh **DELETE** để xóa dữ liệu từ một bảng MySQL là:

```
DELETE FROM ten_bang [Menhde WHERE]
```

- Nếu mệnh đề **WHERE** không được xác định, thì tất cả bản ghi sẽ bị xóa từ bảng MySQL đã cho.
- Bạn có thể xác định bất kỳ điều kiện nào với mệnh đề **WHERE**.
- Bạn có thể xóa các bản ghi trong một bảng đơn tại cùng một thời điểm.

Mệnh đề **WHERE** là thực sự hữu ích khi bạn muốn xóa các hàng đã chọn trong một bảng. Nếu không xác định mệnh đề **WHERE** thì thật sự rất nguy hiểm bởi vì **toàn bộ bảng của bạn sẽ bị xóa**. Do đó bạn phải thật cẩn thận khi sử dụng lệnh này.

Xóa dữ liệu trong MySQL

Ví dụ sau sẽ xóa một bản ghi trong bảng **sinhvienk61** có **mssv** là 4.

```
DELETE FROM sinhvienk61 WHERE mssv=4;
```

Bây giờ, nếu bạn sử dụng lệnh **SELECT** với bảng trên:

```
SELECT * FROM sinhvienk60;
```

Thì kết quả là bản ghi có **mssv=4** đã bị xóa và trong bảng **sinhvienk60** của chúng ta chỉ còn lại 3 bản ghi.

